

Umang Patel

Engineering Project Portfolio

Burnaby, BC | (+1) 236-514-4742 | umang9196@gmail.com

ABOUT THIS DOCUMENT

I am a mechanical engineering master's student at the University of Victoria, finishing in September 2026. I have about two years of industry experience: a year designing hydraulic power systems at Bosch Rexroth India, and a co-op at Trigon Pacific Terminals in Prince Rupert investigating equipment failures on site at a bulk export terminal.

For each project below: what the goal was, what I did, and what actually came out of it. Where the work was a team effort or a result is still unsettled, I say so.

Full project reports, thesis chapters, and source code are available on request.

Updated September 2026 | umang9196@gmail.com | (+1) 236-514-4742

1. CONSTRAINED TRAJECTORY OPTIMIZATION (MASTER'S RESEARCH)

Graduate research, University of Victoria — sole developer

MATLAB with CasADi, Python with Pinocchio and MuJoCo | Jan 2023 to present

The problem. Motion plans for robot arms often route them through configurations where they lose the ability to move or apply force in some directions. Keeping the arm in well-conditioned configurations while still respecting hard limits on joints, velocities, torques and obstacles is the difficult part, because the conditioning is a preference and the limits are not.

What I did. I implemented a primal-dual augmented Lagrangian differential dynamic programming solver from first principles rather than calling an existing optimization library, applied to a 3-DOF planar arm and a 7-DOF Franka Panda. Later versions reduce the KKT system with a Schur complement to deal with ill-conditioning. I added a variable timestep extension that refines the discretization where the arm is near singular, and separate branches implement ADMM-based obstacle avoidance and successive convexification.

The part that mattered most. Most of the work was diagnostic rather than derivational. I built the logging, plotting and comparison tooling used to evaluate solver versions across large batches of runs, because without it I could not tell whether a change in convergence came from the formulation, the numerics, or an implementation error. One example: a gravity compensation error that only appeared in specific joint configurations and was invisible in aggregate convergence statistics until the per-configuration breakdown existed.

Where it stands. The solver runs on both platforms and handles the constraint set. A comparison study across two manipulability indices and two enforcement strategies, hard constraint against soft penalty, is still converging, and the early indication that enforcement strategy matters more than index choice is not something I would defend yet.

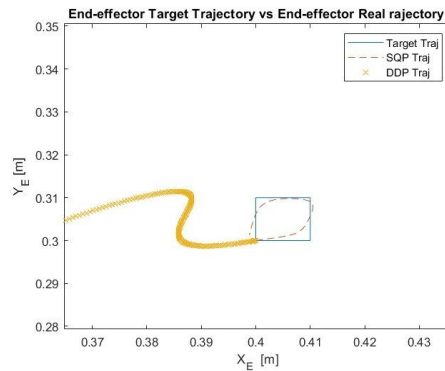
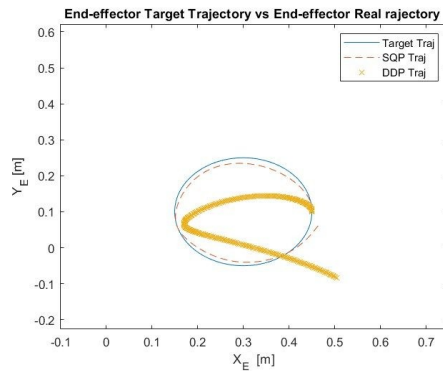
2. SQP AGAINST DDP FOR MANIPULATOR TRAJECTORY TRACKING

MECH 580 Engineering Optimization, University of Victoria — sole author

MATLAB, both solvers written from scratch | December 2023

The problem. Plan joint trajectories for a 2-DOF planar arm that track a target end-effector path, treating the arm's nonlinear dynamics as equality constraints rather than wishing them away.

What I did. I derived the manipulator dynamics from the Lagrangian, discretized them, and set the problem up as a constrained optimization minimizing a quadratic cost on states and inputs with torque limits. I then implemented two solvers myself rather than calling a library: sequential quadratic programming, condensing the horizon into a single QP with the linearized dynamics as equality constraints, and differential dynamic programming with backward and forward passes, a line search and a state-regularized variant. Both were benchmarked against MATLAB's own fmincon SQP across line, circle and square target paths.



Target path against the SQP and DDP solutions, for a circle and a square. The DDP runs drift off the target on both, which is what the comparison was for.

Result. SQP tracked the target more accurately, particularly through `fmincon`, and handled the nonlinear constraints better. My DDP was faster per iteration but less accurate, and on the circle and square it visibly leaves the path. I would not present that as a general verdict on DDP: it says as much about my implementation and the constraint handling as it does about the methods.

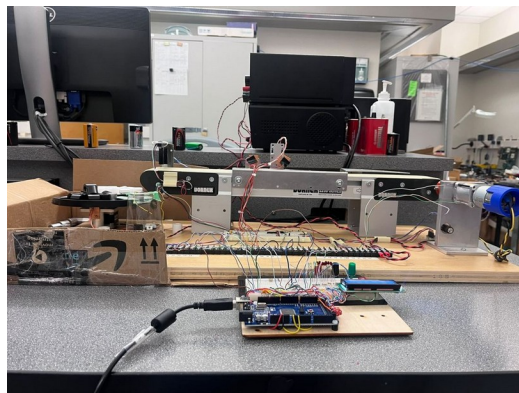
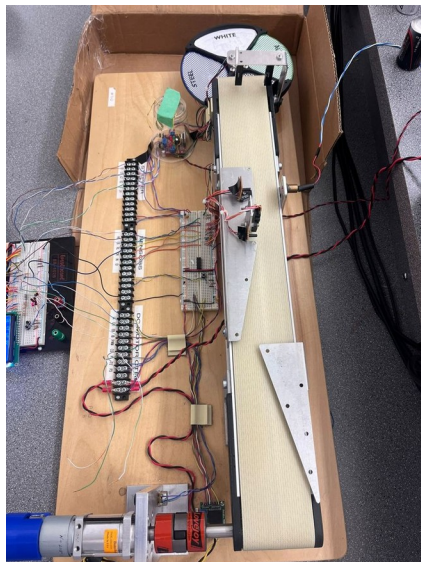
Why it matters here. This project is why my master's research went where it did. Writing both solvers by hand is what made the constrained case, and what to do when constraints and dynamics fight each other, the thing I wanted to work on next.

3. AUTOMATED CONVEYOR SORTING AND INSPECTION SYSTEM

Mechatronics course project — built and tested on hardware

Embedded C on ATmega2560 at 8 MHz | University of Victoria

The problem. Identify and classify items travelling on a conveyor by their material properties, then sort them into the correct bin. The catch is that the sensing happens well before the sorting point, so the system has to remember what each item was while it is still in transit.

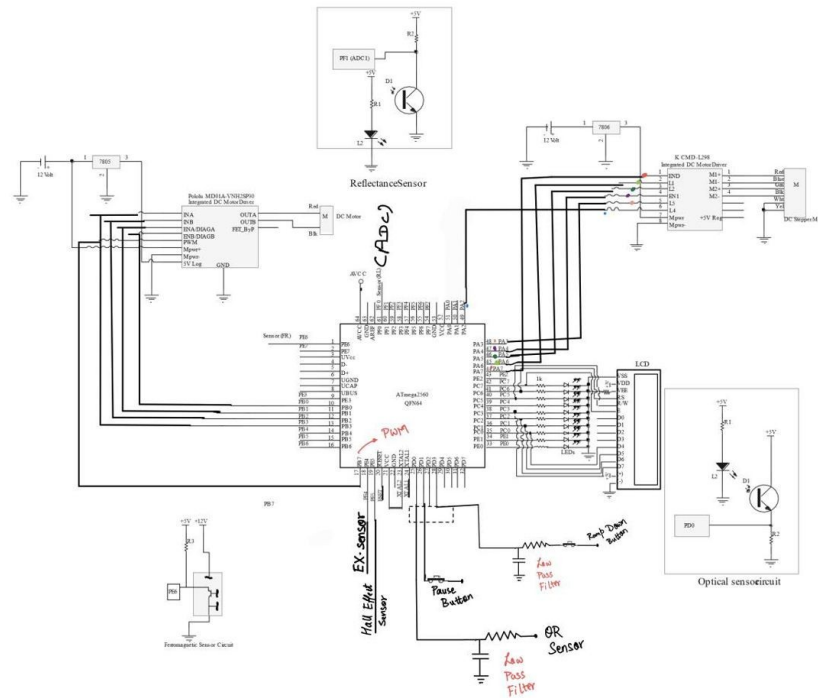


The rig: conveyor, sensor stations, stepper-driven sorting arm, and the terminal strip and breadboard the firmware talks to.

What I did. I wrote the firmware and did the bring-up and debugging on the bench. Three sensor stations read ferromagnetic properties and reflectivity, a stepper-driven bucket does the sorting, a DC conveyor motor runs under PWM, and two push buttons handle pause and ramp-down.

How it works. Item classification runs off interrupt-driven ADC sampling of the reflectance sensor. Classified items go into a linked list used as a FIFO queue, so the system tracks each item between the sensing point and the bins and fires the bucket

at the right moment. The stepper runs an S-curve acceleration profile, which I added after ramping it linearly and finding it lost steps and never reached the speed I needed.



Complete system circuit: ATmega2560, motor drivers, reflectance and Hall effect sensor circuits, LCD, and the pause and ramp-down inputs.

Result. The system sorted 48 items in 35 seconds with no errors.

What I took from it. Nearly all the debugging time went into the timing relationship between the electrical and software sides, not into either alone. Faults that presented as classification errors turned out to be queue and timing errors, and the only way to tell them apart was to instrument the firmware so it reported what it believed was happening. Building the measurement before trusting the result is a habit I have carried into everything since.

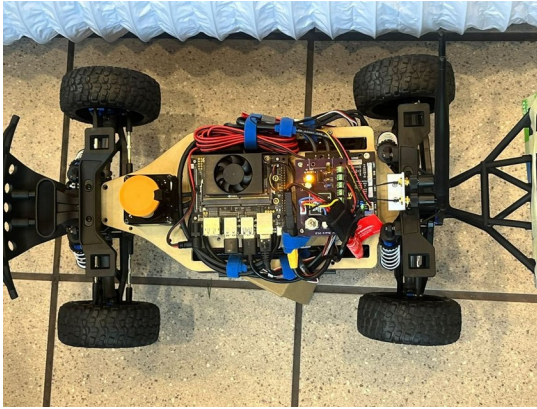
4. F1TENTH AUTONOMOUS VEHICLE

UVic CAV Racing Team, BLUE Team — 2024 AI Velocity Cup champions

NVIDIA Jetson, ROS2 | IEEE Victoria Section, University of Victoria

The project. A one-tenth scale autonomous racing platform running a ROS2 stack on an NVIDIA Jetson, competing in the IEEE Victoria Section AI Velocity Cup: F1Tenth Autonomous Racing Challenge.

What I did. Sensor and actuator integration, wiring and vehicle bring-up, and controller tuning on the physical car. I spent most of my time on the gap between the simulator and the vehicle: sensor noise and latency the simulation did not reproduce meant behaviour that passed in simulation failed on the track. Closing that gap is what actually made the car competitive.



The vehicle, and the Champion Award. Nine-person team; the awards were to the team, not to me individually.

Result. Our team won the Champion Award at the 2024 AI Velocity Cup, and also took the Technical Excellence Award and the Best Team Collaboration Award.

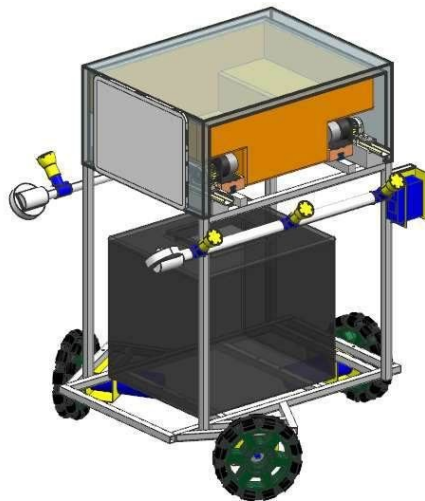
5. MEDICAL AND SANITIZATION ROBOT

B.Eng final year project — four-person team, I led the mechanical design

Siemens NX, Python, Arduino | L.D. College of Engineering | SSIP grant funded

The problem. During COVID-19 there was an urgent need to reduce direct contact between healthcare workers and infected patients. We set out to build a mobile robot that could disinfect surfaces and deliver medication and essentials to patients, so that routine tasks in a ward did not require someone to walk into the room.

What I did. I led the mechanical design: the omnidirectional base, the fluid system for the sanitization module, and the CAD in Siemens NX including structural validation in NX Nastran. My teammates covered the electronics and the control software; I integrated the pumps with the microcontroller for the spraying side.



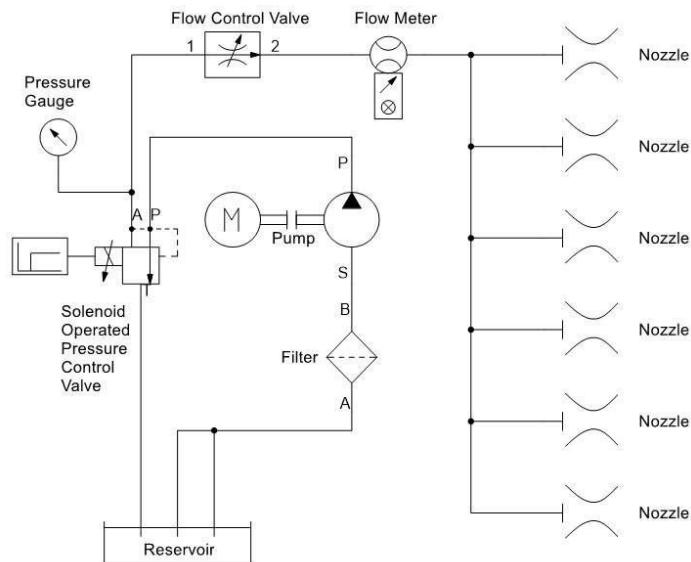
Left: final assembly in Siemens NX. Right: the robot as built, with the delivery bins, control board and spray lines.

Mobility. The robot had to move in confined ward spaces without needing room to turn, so I used a three-wheel holonomic drive. I worked through the inverse kinematics for the three-wheel configuration, sized the wheel diameter and motor requirements against the duty we expected, and selected A1 6061 for the frame.



The fabricated base frame with the three omni wheels and motor mounts.

Sanitization module. A 12 V high-pressure diaphragm pump (8 LPM, 200 PSI cutoff) feeding flat fan spray nozzles from an onboard tank. I designed the circuit around it with a solenoid operated pressure control valve, flow control valve, flow meter, filter and pressure gauge, so the spray could be adjusted rather than run flat out.



Fluid circuit for the sanitization module.

Outcome. The robot was designed, fabricated and demonstrated, and the project received SSIP grant funding. It was a student prototype rather than a hospital-ready device, and the honest limitations were runtime and the robustness of the navigation. Most of the difficulty was packaging the subsystems together: the tank, the pump, the delivery rack and the drivetrain all competing for the same volume and centre of gravity.

6. ODOMETRY ENCODER ASSEMBLY

ROBOCON — three competition seasons

National champion, DD ROBOCON 2019; represented India at ABU ROBOCON 2019, Mongolia

What I designed. Odometry on these robots was only as good as the contact between the encoder wheel and the floor, and any slip put the position estimate out. I designed a spring-loaded mount that keeps the encoder wheel preloaded against the surface through the pivot, so it maintains contact over uneven ground and does not slip. It is a small assembly, but it is the piece where the mechanical design directly determined whether the control worked.



Left: the spring-preloaded encoder mount. Right: the sub-assembly installed on the competition robot.

Summary. Three seasons building competition robots to fixed deadlines with a multidisciplinary team. The 2019 team won the national championship and competed internationally in Mongolia. I was a team member rather than team lead.

7. OTHER WORK

Adaptive model predictive control with online system identification

MATLAB | Individual project

Summary. An MPC degrades as soon as the plant parameters it assumes drift. I put the dynamics in regressor form and ran recursive least squares at every timestep to identify parameters online, feeding the updated model back into the controller. Against a non-adaptive baseline that gave roughly 63% tracking improvement with 0.92% final parameter error, in simulation rather than on hardware.

Sustainable water heating for remote camp sites

MECH 580 Modelling and Simulation of Dynamic Systems | Two-person project, work split evenly

Summary. We modelled a solar-pumped system that stores water as gravity head, runs it back through a turbine and generator, and heats a tank with an immersion heater, using elemental models, linear graphs, and Government of Canada solar and temperature data for the site. The modelling said the design does not work: the storage tank empties in under ten minutes, and at the power the turbine produces it would take roughly 32 hours to raise 28,000 litres by one degree. The individual subsystems behaved sensibly, but the concept as a whole does not close. We reported that rather than tuning the write-up around it, and set out what we would do instead: heat the water directly from the panels and store far less of it.

Human-robot collaboration simulation (UR10e)

MATLAB, Simulink, Python

Summary. I built seven modular MATLAB and Simulink subsystem functions reproducing a published collaborative control scheme, and debugged the accompanying Python simulation, where the substantive fixes were a Procrustes SVD alignment issue and a null-space projector error. Reimplementing someone else's specification and finding where the implementation silently disagrees with it is the same activity as verifying that a system does what its documentation claims.

TOOLS

Siemens NX with NX Nastran, SolidWorks, AutoCAD, MATLAB with Simulink and CasADi, Python with NumPy, Pinocchio and MuJoCo, C for AVR embedded targets, C++ at a developing level, ROS2, Git, Linux, SAP, MASTERCAM with CNC certification from the Indo-German Tool Room.